Universidad de La Sabana Felix Mohr & Sergio Amortegui
Facultad de Ingenería 2026.1

Fundamentos de Inteligencia Artificial

- Sheet 5-

Environment & Rules. Use Python with only `numpy`. Do not use global random state; all randomness must come from a provided `numpy.random.Generator` named `rng`. Unless explicitly stated otherwise, functions should accept probability distributions either as `numpy` arrays or as Python dictionaries mapping outcomes to probabilities (they must be normalized internally). Keep the function signatures from the provided `.py` templates unchanged, as they will be tested automatically on Gradescope. You will only see public test outputs.

Files to implement. `strategy.py`, `rps_strategy.py`. Do not rename files or functions.

Exercise 1 (From probability space to induced distributions – 2 Points) We now work under *risk* (known probabilities), not uncertainty. Consider the finite probability space $(\Omega, \mathbb{P})$ with $\Omega = \{1, \dots, 10\}$ and the uniform measure $\mathbb{P}(\{\omega\}) = \frac{1}{10}$ for each $\omega \in \Omega$. Define the random variable $X : \Omega \to \{0, 1, 2\}$ by

$$X(\omega) = \begin{cases} 0 & \text{if } \omega \in \{1,2,3,4,5\} \quad \text{(Rock)}, \\ 1 & \text{if } \omega \in \{6,7,8,9\} \quad \text{(Paper)}, \\ 2 & \text{if } \omega \in \{10\} \quad \text{(Scissors)}. \end{cases}$$

You may represent X as a dictionary or as an integer array mapping each $\omega \in \{1, \dots, 10\}$ to $\{0, 1, 2\}$.

Implement `compute_distribution(origin, mapping)` that returns the *induced* discrete distribution of X, i.e. the vector $\big(\mathbb{P}(X = 0), \mathbb{P}(X = 1), \mathbb{P}(X = 2)\big)$. Here, `origin` is the base probability measure on Ω given as dict `{omega: prob}`; and `mapping` is X given a dict `{o: k}` where k is the set of possible values for an outcome $o \in \{1, 2, 3\}$.

Your function must normalize inputs and return a length-3 array with nonnegative entries summing to 1.

Exercise 2 (Evaluating actions by expectation – 2 Points) Given the opponent's move distribution and a payoff table, implement

`expected_payoff(opponent_dist, payoff_table).`

`opponent_dist` is the *induced* distribution over opponent outcomes $o \in \{0, 1, 2\}$ (Rock, Paper, Scissors), given as a length-3 array. `payoff_table` is a 3×3 array with entries $\text{payoff}(a, o)$ (rows are our actions a, columns the opponent outcomes o).

Return a length-3 array with the expected payoff of each action:

$$\forall a \in \{0, 1, 2\} : \quad \mathbb{E}[\text{payoff} \mid a] = \sum_{o=0}^{2} p(o)\, \text{payoff}(a, o).$$

(Under risk, $p(o)$ does not depend on our action.)

Exercise 3 (Acting by expected utility – 3 Points) To account for risk attitudes, actions should be compared by *expected utility*. Implement

```
expected_utility(opponent_dist, payoff_table, utility_fn).
```

This function is identical in structure to Exercise 2 but applies a user-supplied utility function `utility_fn` to the payoff entries before taking expectations:

$$\forall a: \quad \mathbb{E}[u(\text{payoff}) \mid a] = \sum_{o=0}^{2} p(o)\, u\big(\text{payoff}(a, o)\big).$$

`utility_fn` is a callable (e.g., log-utility, CRRA, or risk-neutral $u(x) = x$). Return a length-3 array of expected utilities.

Exercise 4 (A Rock–Paper–Scissors strategy under risk – 3 Points) We now build a strategy that acts under a *known* opponent distribution (risk, not uncertainty). In `strategy.py`, define an abstract base class `Strategy` with an abstract method

```
decision(self, rng) -> int.
```

In `rps_strategy.py`, implement `RPSStrategy` with constructor

```
__init_(self, utility_fn, mode='expected_utility', origin=None, mapping=None).
```

`origin` & `mapping` If not provided, assume the example from Exercise 1. `utility_fn` is a callable as in Exercise 3. `mode` is either `'expected_payoff'` or `'expected_utility'`.

The per-round payoff is $+1$ for a win, 0 for a tie, and -1 for a loss. Let the payoff table be

$$\begin{pmatrix} 0 & -1 & +1 \\ +1 & 0 & -1 \\ -1 & +1 & 0 \end{pmatrix}$$

with rows indexed by our action ($0 = \text{Rock}, 1 = \text{Paper}, 2 = \text{Scissors}$) and columns by the opponent move.

Behavior. On each call to `decision(rng)`, the strategy:

1. Forms action values by either:
   - computing `expected_payoff(opponent_dist, payoff_table)` if `mode='expected_payoff'`, or
   - computing `expected_utility(opponent_dist, payoff_table, utility_fn)` if `mode='expected_u`
2. Selects an action that maximizes the chosen criterion; in case of ties, break ties *randomly using the provided* ***`rng`*** (no global state).

Return the chosen action as an integer in $\{0, 1, 2\}$.

Exercise 5 (From distributions to decision making – Practice Problem) Consider the finite probability space $\Omega = \{1, 2, 3, 4\}$ with the uniform measure $\mathbb{P}(\{\omega\}) = \frac{1}{4}$ for each $\omega \in \Omega$. Define the random variable $X : \Omega \to \{0, 1\}$ by

$$X(\omega) = \begin{cases} 0 & \text{if } \omega \in \{1, 2, 3\}, \\ 1 & \text{if } \omega = 4. \end{cases}$$

(a) Compute the induced distribution of X, i.e. $\mathbb{P}(X = 0)$ and $\mathbb{P}(X = 1)$. (1 Point)

(b) Suppose the agent can choose between two actions $a \in \{A, B\}$. The payoff depends on the outcome of X as follows:

	$X = 0$	$X = 1$
A	1	-1
B	0	2

Compute the *expected payoff* of each action. Which action maximizes the expected payoff?

(c) Now consider the utility function $u(x) = \sqrt{x + 2}$, which is well-defined for the payoffs in the table above. Compute the *expected utility* of each action and decide which action is preferred under expected utility. Does the choice differ from part (b)?

(d) Briefly explain why using a utility function can break a tie in expected payoff.